

🖥️ UB 底层架构与工程研讨 (大屏演示版)

开源 KV Cache 架构设计与量化分析

深度拆解 vLLM、Mooncake、SGLang 与 LMCache 的内存池管理、微观 Gather/Scatter 算子、数据打包切片与硬件传输推导

📦 内存池与介质注册

⚡ GPU 向量化 Gather/Scatter

🧩 PCIe / GDR / RoCEv2 量化模型

框架	内存池管理者与类型	内存注册与分配机制 (System API / CUDA API)	存储与传输粒度	回收与淘汰策略
vLLM	CPUAllocator / CacheEngine	启动时执行 <code>torch.empty(..., pin_memory=True)</code> 预分配锁页内存。HBM 与 DDR 保持 1:1 物理 Block 镜像分配。	Block 16 / 32 Tokens	Free-list 块索引栈，无碎片；依靠 CUDA 异步 Stream 执行 <code>swap_in</code> / <code>swap_out</code> 2D/3D Memcpy。
Mooncake	SegmentManager / MooncakeStore	预申请 HugePages (2MB/1GB) 或 <code>posix_memalign</code> ，调用 <code>ibv_reg_mr</code> 注册为 RDMA MR，支持 <code>cudaHostRegister</code> 直连。	Slice / Segment 1MB / 4MB / 16MB	Slab Allocator / CacheLib 算法，支持全局多副本去热点与 LRU/LFU 动态 Slice 淘汰。
SGLang	ReqToTokenPool / HiCacheManager	GPU HBM 划分为二维 Slot Pool；DDR / SSD 通过 HiCache 映射。SSD 接入支持 <code>io_uring</code> / SPDK 绕过内核 Page Cache。	Token Slot & Radix Node 动态前缀树节点	RadixTree 前缀树匹配，基于 Tree Node 引用计数与 LRU-LFU 递归剪枝淘汰，异步流水线 Prefetch。
LMCache	L1CacheManager / TensorPool	启动根据 <code>--l1-size-gb</code> 一次性分配 CPU Pinned Tensor。封装双缓冲区 (Double-Buffering) 异步数据管道。	Chunk 默认 256 Tokens	全局 Chunk 哈希表 + LRU/LFU 双向链表；GPU 侧调起向量化 <code>LDG/STG.128</code> 算子重组。

Pinned Memory (锁页内存) 的必要性

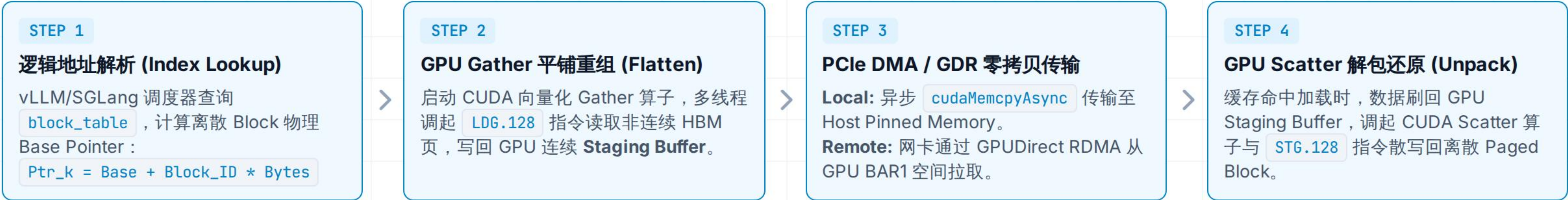
DMA 控制器传输数据依赖物理连续且固定的 Page 地址。未锁页内存页可能触发 Page Fault 被内核 Page Out，导致 PCIe DMA 降级为极其缓慢的软件 CPU Memcpy 拷贝。

HugePages (巨页) 降低 TLB Miss

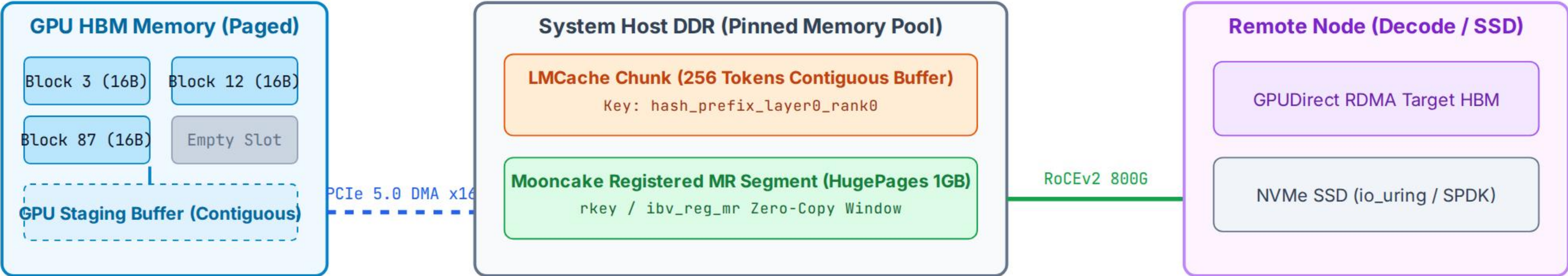
Mooncake 大规模映射 100GB+ Host DDR 时，使用 2MB/1GB HugePages 可将 TLB 页表项减少 512~262144 倍，大幅降低高并发 RDMA 频繁寻址带来的 CPU TLB Miss 开销。

KV Cache 离散 Block 到连续 Chunk 的完整微观传输链路

Pipeline Architecture



硬件层物理数据流动示意图 (GPU HBM ↔ Staging Memory ↔ Host DDR ↔ Network)





🔄 核心矛盾与适配层 (Connector/Adapter) 职责

- **vLLM 侧逻辑:** 感知模型结构, 管理离散 Block ID 映射表 (如请求 A 对应 [Block 3, Block 12, Block 87])。
- **Mooncake 侧逻辑:** 二进制盲操作 (Byte-Blind), 仅接受连续 Base Pointer 与 Length, 提供极致 RDMA 搬运。
- **转译过程:** 由 Worker 内部 Connector 根据物理尺寸公式计算绝对指针, 调起 CUDA Gather 算子或构建 `ibv_sge` 描述符列表。

🔑 Key 命名契约与 Tensor Parallel (TP) 天然隔离

- **结构化 Key 命名规则:**
`Key = Hash(Tokens_0..N) + "_layer_" + Layer_ID + "_rank_" + Rank_ID`
- **TP 物理隔离:** 在 TP 并行中, 每个 GPU Worker 运行在独立进程中, 只处理属于自身 Head 切片的 KV 数据。
- **零跨卡 NVLink 开销:** 各 Rank 独立生成带有 `Rank_ID` 的全局 Unique Key, 并发向 DDR/Remote 读写, 加载阶段无跨卡 All-Gather。

> 伪代码剖析: Connector 地址解析与 Gather 算子调用

```
# 1. 计算单 Block 物理字节数 (仅包含当前 TP Rank 的 Head 数) block_bytes = block_size * 2 * num_layers * (num_heads / tp_size) * head_dim * element_bytes
# 2. 将离散 Block ID 转为物理 GPU 显存绝对指针数组 block_ptrs = [gpu_memory_base + block_id * block_bytes for block_id in block_table]
# 3. 调起 CUDA 向量化 Gather 算子, 平铺写回 GPU 连续 Staging Memory launch_gather_kernel(block_ptrs, staging_buffer_ptr, block_size, num_heads_per_rank, head_dim)
# 4. 交付二进制 Slice 给 Mooncake Store 存取 (完全不感知 Token 语义) mooncake_store.put(key=f"{seq_hash}_l{layer}_r{rank}", src_ptr=staging_buffer_ptr, size=total_bytes)
```


🔧 vLLM: Block Swap 与 Pinned Memory 镜像

- **1:1 块镜像结构**: 启动时预分配 CPU Pinned Blocks，物理结构与 GPU HBM Paged Block 保持严格 1:1 对齐。
- **Swap-In / Swap-Out 流程**: 显存不足时，调度器将 Block 编号传给 `swap_out`，调起 `cudaMemcpy2DAsync` 执行 2D/3D 张量页拷贝。
- **缺陷分析**: 按 16/32 Tokens 小 Block 频繁发起 API 调用，产生 CUDA Launch Overhead (约 3~5μs)，导致 PCIe 队列饥饿，带宽利用率仅 ~45%。

🌲 SGLang: RadixTree 层次匹配与 HiCache 三级下沉

- **RadixTree 细粒度索引**: 以 Token 序列作为树枝路径。匹配 Prefill 时可精确重用任意长度的前缀节点。
- **HiCache 三级缓存 (L1/L2/L3)**:
 - **L1 (GPU HBM)**: 驻留当前推理 Batch 热 Block。
 - **L2 (Host DDR)**: 驱逐前缀挂载至 CPU 节点。
 - **L3 (NVMe/Remote)**: 通过 `io_uring` / SPDK 刷入 SSD，绕过 Page Cache。
- **Chunked Prefill 流水线**: 将传输时间隐藏在 Prefill 计算重叠期。

📦 SGLang HiCache 节点状态转换与 SPDK I/O 路径

L1 HBM HOT

Radix Node: Evict Triggered

显存满，选择 LRU 叶子节点，标记为 `IN_TRANSFER`，锁定节点防止被 GC 误删。



CUDA GATHER

HBM → CPU Pinned

调起 `gather_paged_kv` CUDA 内核平铺写入 Host DDR，更新树节点指针指向 L2 内存。



L3 SPDK I/O

CPU DDR → Local NVMe

SPDK / `io_uring` 写入 Submission Queue，NVMe 控制器通过 PCIe DMA 直接落地 Flash。

🧩 Mooncake: GPUDirect RDMA (GDR) 与 Group Slice

- **PD 分离架构核心:** 专为 Prefill-Decode 解耦设计。Prefill 节点生成 KV Cache 后直接通过网络推送至 Decode 节点 HBM。
- **GPUDirect RDMA 零拷贝:** NIC 充当 PCIe Master，向 GPU BAR1 空间直接发起 P2P Read/Write，**完全 Bypass Host CPU 与 Host DDR**。
- **Group Slice 多副本去热点:** 将连续大 Segment 拆分为 1MB/4MB Slice，按 Hash 分布在不同 Storage 节点，消除读倾斜。

🧩 LMCache: 256-Token Chunking 与双缓冲流水线

- **256-Token Chunk 抽象:** 将 16 个 vLLM 离散 Block (256 Tokens) 聚合成一个 Chunk，降低元数据索引量 16 倍，大幅提高 PCIe 载荷。
- **Vectorized CUDA Kernel:** 采用 `LDG.128` / `STG.128` 向量化指令（每次读取 16 Bytes），极致压榨 GPU 显存带宽。
- **Double-Buffering Pipeline:** Layer $L+1$ 执行 GPU Gather 的同时，Layer L 数据已通过异步 Stream 在 PCIe 上传输，实现极度重叠。

</> LMCache 向量化 128-bit CUDA Gather Kernel 微观实现机理

```
// CUDA Vectorized 128-bit (16 Bytes) Gather Kernel Excerpt __global__ void gather_paged_kv_kernel( const uint4* __restrict__ paged_kv_base,
// 128-bit aligned Paged KV Source uint4* __restrict__ contiguous_staging, // Continuous Output Buffer const int32_t* __restrict__
block_table, int block_size, int head_dim) { // 利用 uint4 (16 Bytes) 实现单指令 128 位向量化合并访存 (Coalesced Access) int tid = blockIdx.x *
blockDim.x + threadIdx.x; int block_idx = tid / (block_size * head_dim / 8); // 16B per load int physical_block = block_table[block_idx];
uint4 val = __ldg(&paged_kv_base[physical_block * offset + intra_offset]); contiguous_staging[tid] = val; // 连续平铺写入 Staging Buffer }
```


🔗 PCIe TLP 事务包损耗

PCIe 5.0 x16 理论带宽 64GB/s。每个 TLP 数据包包含：

- Header: 12-16 Bytes
- Sequence + LCRC: 6 Bytes
- Framing Overhead: 2 Bytes

结论: 小包 (256B) 效率仅 **91.4%**；以 4KB (MPS) 粒度 Chunk 传输，TLP 效率提升至 **98.5%+**。

📦 RoCEv2 MTU 封包率

Mooncake 使用 4096B MTU 传输二进制 Slice，报文格式：

- ETH + IP + UDP Header: 42B
- IB BTH + ICRC: 16B
- Payload: 4096 Bytes

协议开销: Header 仅占 1.4%，网络有效吞吐达到理论极限 **98.6%**。

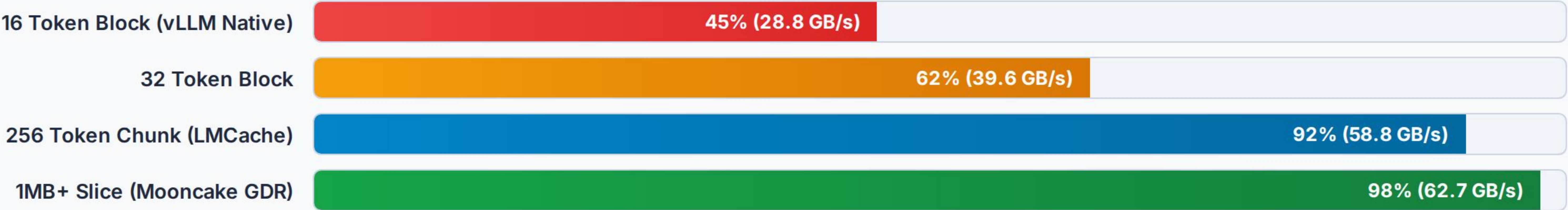
⚠️ NUMA 跨 Socket 衰减

若 GPU 在 Socket 0，而 CPU Pinned DDR 在 Socket 1：

- 数据必须跨越 Intel UPI / AMD xGMI 链路。
- 引入额外 100ns+ 延迟与总线拥塞。

吞吐惩罚: 跨 NUMA 传输引入 **25%~40% 带宽衰减**，需用 `numactl --membind` 绑定。

📊 不同传输粒度下的 PCIe 有效利用率对比 (以 PCIe 5.0 x16 64GB/s 为例)



√ 1. KV Cache 单卡物理数据量公式

$$S_{kv}^{(TP)} = 2 \times N_{token} \times L_{layer} \times \frac{H_{kv}}{TP} \times D_{head} \times B_{elem}$$

其中 2 代表 Key/Value 两组张量， $B_{elem} = 2$ (FP16/BF16)。若开启 FP8/INT8 量化则乘以 1。

🕒 2. 端到端传输延迟重叠模型 (Pipeline Overlapping)

$$T_{total} = T_{meta} + \max(T_{gather}, T_{xfer_bus}, T_{scatter}) + T_{sync}$$

其中 $T_{xfer_bus} = \frac{S_{kv}^{(TP)}}{BW_{effective}}$ 。优化目标是让 CUDA Gather/Scatter 与 PCIe/RDMA 总线传输完全隐藏。

📊 单 Token 物理尺寸计算标尺 (以 Llama-3-70B, TP=8, FP16 为例)

- **模型结构:** $L_{layer} = 80$, $H_{kv} = 8$ (GQA), $D_{head} = 128$, FP16 ($B_{elem} = 2$ Bytes)。
- **全模型单 Token KV 尺寸:** $2 \times 80 \times 8 \times 128 \times 2 = 327,680$ Bytes ≈ 320 KB/Token。
- **TP=8 单卡承载量:** 由于 TP=8，每卡分摊 1 个 KV Head，单卡单 Token KV 尺寸为 **320 KB/Token/Rank**。

核心推导公式：所需有效物理带宽 BW_Req = 单卡数据量 (S_kv_rank) / 业务容忍传输时延预算 (T_budget)
(基准模型：Llama-3-70B, TP=8, 单卡 320 KB/Token/Rank)

业务场景类型	并发 (Batch) & 序列长度 (Seq)	单卡传输 KV 数据量	传输时延预算 (TTFT/TPOT)	推导有效物理带宽及硬件拓扑诉求
高并发 短上下文 (在线对话/API 交互)	Batch = 128 Seq Len = 2,048 Tokens	320KB × 128 × 2048 = 83.88 GB	200 ms (严格 TPOT 限制 Swap In)	83.88 / 0.2 = 419.4 GB/s 诉求: 常规 PCIe/RDMA 溢出，必须依赖 NVLink/NVSwitch 节点内 P2P。
中并发 长上下文 (文档问答/RAG 检索)	Batch = 16 Seq Len = 32,768 Tokens (32K)	320KB × 16 × 32768 = 167.77 GB	1500 ms (TTFT 首字延迟容忍较宽)	167.77 / 1.5 = 111.8 GB/s 诉求: 需配置双 PCIe 5.0 x16 通道 (128GB/s) 或 Multi-rail 2×400G RDMA。
极端长文本 PD 分离 (超长分析/标书读取)	Batch = 1 Seq Len = 131,072 Tokens (128K)	320KB × 1 × 131072 = 41.94 GB	500 ms (Prefill 向 Decode 搬运容忍)	41.94 / 0.5 = 83.88 GB/s 诉求: 单轨 800Gbps RDMA (约 100GB/s) 或 NDR IB (GDR 零拷贝) 刚好打满。
代码生成与 多轮 Agent (复杂多步工具调用)	Batch = 32 Seq Len = 8,192 Tokens (8K)	320KB × 32 × 8192 = 83.88 GB	800 ms (高响应度交互要求)	83.88 / 0.8 = 104.85 GB/s 诉求: 需 PCIe 5.0 x16 全速总线 + 400G RDMA 组网。

💡 硬件选型启示

对于大模型推理节点，网络网卡 (NIC) 与 PCIe 树拓扑的配比直接由 KV Cache 的传输吞吐决定。在部署 PD 分离与长文本业务时，网卡带宽必须按 GPU 显存 DMA 极限 (如 PCIe 5.0 64GB/s 或 800G RDMA) 按 1:1 对等组网。

⚙️ 软件架构演进关键点

- **摒弃小 Block 传输:** 必须引入类似 LMCache 的 **Chunking 机制 (128-256 Tokens)** 或 Mooncake 的 **Slice 聚合**，消除 TLP 头开销与 API 启动延迟。
- **向量化 Gather/Scatter 算子:** 必须使用 CUDA `LDG.128` / `STG.128` 指令做 16 字节合并访存，实现 GPU 显存带宽饱和。
- **双缓冲流水线 (Double-Buffering):** 将 GPU Gather 算子计算、PCIe 传输、磁盘/网络 I/O 在 Layer 维度高度 Pipeline 重叠。

🔌 底层硬件与 NIC 设计诉求

- **硬件级 Scatter-Gather DMA:** 推动网卡 (NIC) 与 PCIe DMA 控制器原生支持 Scatter-Gather 描述符列表，实现真正的 GPU 显存零拷贝直接 RDMA。
- **GPUDirect P2P Bypass:** 拓扑设计上压制跨 NUMA 访问，利用 GPUDirect RDMA 直接打通 GPU BAR1 与网卡，彻底 Bypass CPU 与 Host DDR。
- **全链路物理带宽匹配:** PCIe 5.0/6.0 与 800G/1.6T RoCEv2 网络是缓解千亿模型高并发下 KV 传输瓶颈的基石。

致谢 & Q/A 技术讨论

开源 KV Cache 架构设计与量化分析 | UB 底层开发团队内部研讨